

This document contains the **first 10 pages** and **last 10 pages** of the application's source code, concatenated in a single fixed, documented order across **69 real source files** (backend Python + frontend TypeScript/React) — **284 pages** in total at 46 lines/page. Per Copyright Office practice for a representative excerpt, the middle 264 pages are omitted from this document (not silently skipped — this cover states exactly how many, and the full pagination manifest used to select these 20 pages is available on request). Each page below carries a running header showing the exact file it came from and its position within that file, plus a footer showing its position in the full 284-page concatenation.

Full file order used for this concatenation (page 1 begins at the top of the first file listed; the last page ends at the bottom of the last file listed):

1. backend/app/main.py
2. backend/app/ml/engine.py
3. backend/app/ml/llm_features.py
4. frontend/src/app/page.tsx
5. frontend/src/app/graph/page.tsx
6. frontend/src/lib/AuthContext.tsx
7. frontend/src/lib/api.ts
8. frontend/src/components/AntaraLoader.tsx
9. frontend/src/components/AntaraMark.tsx
10. frontend/src/components/AntaraWordmark.tsx
11. frontend/src/components/AppBootGate.tsx
12. frontend/src/components/ArchetypeSheet.tsx
13. frontend/src/components/BudgetSheet.tsx
14. frontend/src/components/BurnGauge.tsx
15. frontend/src/components/CategoryDetailSheet.tsx
16. frontend/src/components/CategoryDistributionChart.tsx
17. frontend/src/components/CategoryIcon.tsx
18. frontend/src/components/ConsentGate.tsx
19. frontend/src/components/CountUpNumber.tsx
20. frontend/src/components/DataConfigPanel.tsx
21. frontend/src/components/InstancesSheet.tsx
22. frontend/src/components/LearningCurveSheet.tsx
23. frontend/src/components/LegalPageLayout.tsx
24. frontend/src/components/MobileFrame.tsx
25. frontend/src/components/NewUserOnboardingSheet.tsx
26. frontend/src/components/PageTransition.tsx
27. frontend/src/components/PopulationDotGraphCanvas.tsx
28. frontend/src/components/PublicSignupToggle.tsx
29. frontend/src/components/PullCanvas.tsx
30. frontend/src/components/QuickLogSheet.tsx
31. frontend/src/components/SampleSizeTrend.tsx
32. frontend/src/components/StreakBadge.tsx
33. frontend/src/components/SuperadminPanel.tsx
34. frontend/src/components/TransactionEditSheet.tsx
35. frontend/src/components/WhatsNewGate.tsx
36. frontend/src/components/WhatsNewSheet.tsx
37. frontend/src/components/WhyPredictionSheet.tsx
38. frontend/src/components/survey/AntaraBootloader.tsx
39. frontend/src/components/survey/AntaraMark.tsx
40. frontend/src/components/survey/AntaraWordmark.tsx
41. frontend/src/components/survey/ChoiceList.tsx
42. frontend/src/components/survey/NumericKeypad.tsx
43. frontend/src/components/survey/StepContainer.tsx
44. frontend/src/components/survey/StepFooter.tsx
45. frontend/src/components/survey/SuccessBurst.tsx
46. frontend/src/components/survey/SurveyProgress.tsx
47. frontend/src/lib/brand.ts
48. frontend/src/lib/changelog.ts
49. frontend/src/lib/constants.ts
50. frontend/src/lib/firebase.ts
51. frontend/src/lib/surveyApi.ts
52. frontend/src/lib/surveyConstants.ts
53. frontend/src/app/admin/page.tsx
54. frontend/src/app/admin/training-insights/page.tsx
55. frontend/src/app/chat/page.tsx
56. frontend/src/app/layout.tsx
57. frontend/src/app/loading.tsx
58. frontend/src/app/privacy/page.tsx
59. frontend/src/app/review/layout.tsx
60. frontend/src/app/review/page.tsx
61. frontend/src/app/terms/page.tsx
62. frontend/src/types/index.ts
63. frontend/src/types/survey.ts
64. frontend/src/tests/firestore-rules.test.ts
65. backend/app/firebase_admin.py
66. backend/app/ml/__init__.py
67. backend/app/ml/ollama_client.py
68. backend/app/ml/survey_etl.py
69. backend/app/schemas.py

```
import os
import uvicorn
from fastapi import FastAPI, Depends, HTTPException, Query, Body
from fastapi.middleware.cors import CORSMiddleware
from typing import List, Optional, Dict, Any
from datetime import datetime

from app.schemas import (
    SpendPredictRequest, SpendPredictResponse,
    DotGraphResponse, HealthResponse, TransactionItem,
    CategorizeRequest, CategorizeResponse,
    InsightRequest, InsightResponse,
    ChatRequest, ChatResponse,
    LearningCurveResponse,
    AllocateBudgetRequest, AllocateBudgetResponse,
)
from app.ml.engine import MLEngine
from app.ml import survey_etl, llm_features, ollama_client
from app.firebase_admin import (
    initialize_firebase_admin, verify_firebase_token,
    require_superadmin, get_firestore_client
)

app = FastAPI(
    title="Antara ML Spend Prediction & Personalization API",
    version="1.0.0",
    description="FastAPI service for expense prediction, teen spend embeddings, and Obsidian dot graph physics"
)

# CORS Middleware config
#
# Step 9 (2026-08-21): switched from a wildcard ("*") to an explicit allowlist.
# This isn't just tidying – `allow_origins=["*"]` combined with
# `allow_credentials=True` is invalid per the CORS spec (browsers refuse a
# literal "*" origin on credentialed requests) and only ever "worked" because
# every real browser call up to now went through the Next.js rewrite proxy
# (next.config.js), making it same-origin from the browser's point of view –
# CORS was never actually exercised. Step 9 points the frontend at
# https://api.antara.money directly (see frontend/src/lib/api.ts's
# API_BASE_URL), making this a genuine cross-origin call for the first time,
# so the allowlist needs to be real. Keeps every existing access path – the
# Tailscale IP, LAN IP, and local dev – none of that narrows.
ALLOWED_ORIGINS = [
    # Production domain (Step 9) – app.antara.money is the frontend; the bare
    # apex is included too in case anything ever serves directly from it.
```

```
"https://app.antara.money",
"https://antara.money",
# Tailscale (stable, private) – unchanged fallback access path.
"http://100.103.94.116:3001",
"http://100.103.94.116:8001",
# LAN – unchanged fallback access path.
"http://192.168.0.8:3001",
"http://192.168.0.8:8001",
# Local dev.
"http://localhost:3000",
"http://localhost:3001",
"http://127.0.0.1:3000",
"http://127.0.0.1:3001",
]

app.add_middleware(
    CORSMiddleware,
    allow_origins=ALLOWED_ORIGINS,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.on_event("startup")
async def startup_event():
    initialize_firebase_admin()
    # Step 10: load whatever Stage-1 survey-derived benchmarks were last
    # computed (admin/categoryBenchmarks) and overlay them onto the cold-start
    # heuristic's static fallback numbers. If nothing's been computed yet
    # (fresh deploy, or Firestore unreachable), this safely no-ops and the
    # Step 8 static numbers keep serving – never a startup failure either way.
    try:
        db = get_firestore_client()
        existing = survey_etl.load_benchmarks(db)
        if existing:
            applied = MLEngine.apply_live_benchmarks(existing)
            print(f"[+] Loaded existing survey benchmarks (n={existing.get('sampleSize')}), applied={
{applied}}")
        else:
            print("[i] No admin/categoryBenchmarks doc yet – run POST /api/v1/admin/recompute-benchm
arks once survey data exists")
    except Exception as e:
        print(f"[!] Warning: could not load survey benchmarks at startup: {e}")
    print("[+] Antara ML API service initialized on port 8001")

@app.get("/health", response_model=HealthResponse, tags=["System"])
```

```

@app.get("/api/v1/health", response_model=HealthResponse, tags=["System"])
async def health_check():
    return HealthResponse(
        status="healthy",
        service="antara-ml-backend",
        port=8001,
        version="1.0.0",
        timestamp=datetime.utcnow()
    )

@app.post("/api/v1/predict/spend", response_model=SpendPredictResponse, tags=["ML Prediction"])
async def predict_spending(
    req: SpendPredictRequest,
    current_user: dict = Depends(verify_firebase_token)
):
    """
    Computes forecasted categorical spend, burn rate, risk levels, and smart teen insights.
    Caches the forecast to Firestore `predictions/{uid}/{period}` when Firestore is reachable.

    Revived in Step 8 (2026-08-21) – this is what powers the "Why" screen reachable from
    BurnGauge's coaching line on the frontend (see frontend/src/components/WhyPredictionSheet.tsx
    and lib/api.ts's fetchSpendPrediction). Cold-start heuristic vs. trained-embedding mode is
    decided by MLEngine based on logged history, not guessed client-side – see `is_cold_start` /
    `model_mode` on the response, which the frontend labels honestly rather than presenting a
    heuristic guess as a confident prediction.
    """
    prediction = MLEngine.calculate_spend_predictions(
        user_id=req.user_id,
        transactions=req.transactions,
        monthly_budget=req.monthly_budget or 5000.0,
        period_days=req.period_days or 30
    )

    # Cache prediction in Firestore if available
    db = get_firestore_client()
    if db:
        try:
            period_key = datetime.utcnow().strftime("%Y-%m")
            pred_doc = db.collection("predictions").document(req.user_id).collection("periods").document(period_key)
            pred_doc.set(prediction.dict(), merge=True)
        except Exception as e:
            print(f"[!] Warning: Could not write prediction cache to Firestore: {e}")

    return prediction

```

```
@app.post("/api/v1/ml/dot-graph", response_model=DotGraphResponse, tags=["Personalization"])
async def get_dot_graph(
    req: SpendPredictRequest,
    current_user: dict = Depends(verify_firebase_token)
):
    """
    Computes learned spend embedding space and generates Obsidian-style force-directed
    graph nodes (user core, category gravity centers, peer archetypes) and physics springs.

    Not currently called by the frontend – the Pull screen (frontend/src/components/
    PullCanvas.tsx) uses its own pure client-side two-pole need/want layout, and Step 8's
    ML revival scoped the frontend wiring to the "Why" prediction screen only (predict_spending
    above), not this dot-graph. Left fully functional and unretired (no dormant-by-design
    notice) since Step 8 reversed that framing – it just doesn't have a UI consumer yet.
    """
    return MLEngine.generate_dot_graph(
        user_id=req.user_id,
        transactions=req.transactions
    )

@app.post("/api/v1/ml/learning-curve", response_model=LearningCurveResponse, tags=["Personalization"])
async def get_learning_curve(
    req: SpendPredictRequest,
    current_user: dict = Depends(verify_firebase_token)
):
    """
    A real, per-user confidence-over-time curve for the Pull screen – not an
    illustrative chart. Replays MLEngine's own confidence formula against the
    caller's own real logged days (see MLEngine.calculate_learning_curve),
    the same formula predict_spending uses for "right now," just walked
    across their actual history so the curve is genuinely theirs, not a
    generic shape every user would see. Same request/auth shape as
    /predict/spend and /ml/dot-graph – transactions are supplied by the
    caller (already the client's own data), not re-fetched from Firestore.
    """
    points = MLEngine.calculate_learning_curve(transactions=req.transactions)
    return LearningCurveResponse(user_id=req.user_id, points=points)

@app.post("/api/v1/ml/allocate-budget", response_model=AllocateBudgetResponse, tags=["Personalization"])
async def allocate_budget(
    req: AllocateBudgetRequest,
    current_user: dict = Depends(verify_firebase_token)
):
    """
```

```
"Instances" – the user pins exact amounts to whichever categories they
choose; this computes a real suggested split of whatever's left across
everything else, proportional to that category's own real historical
spend (see MLEngine.allocate_budget for the full fallback chain and
staged-honesty flagging). Same request/auth shape as /predict/spend –
the caller supplies its own already-fetched transactions.
"""

result = MLEngine.allocate_budget(
    transactions=req.transactions,
    monthly_budget=req.monthly_budget,
    pinned=req.pinned,
)
return AllocateBudgetResponse(user_id=req.user_id, **result)

def _require_self_or_superuser(current_user: dict, user_id: str) -> None:
    """Same access boundary as everywhere else a request carries a
    `user_id`: a valid token alone isn't enough, it has to be *that* user's
    own token (or a superadmin's). Firestore rules already enforce this for
    direct client reads of `users/{uid}/transactions` (see firestore.rules);
    this backend reads that same collection via the Admin SDK, which
    bypasses rules entirely, so this check is what actually stands in for
    them here – the point being no new bypass path is introduced by the
    Ollama-backed endpoints below just because the SDK could technically
    read anyone's data."""
    if current_user.get("uid") != user_id and current_user.get("role") != "superadmin":
        raise HTTPException(status_code=403, detail="Cannot access another user's data")

@app.get("/api/v1/admin/status", tags=["Superadmin"])
async def get_admin_status(admin_user: dict = Depends(require_superuser)):
    """
    Protected superadmin endpoint to retrieve backend telemetry and model status.
    """
    ollama_reachable, ollama_error = ollama_client.health()
    return {
        "status": "operational",
        "superadmin_email": admin_user.get("email"),
        "port": 8001,
        "ports_guarded": [5000, 8000],
        "systemd_unit": "antara-ml.service",
        "tunnel_configured": True,
        "live_survey_benchmarks_applied": MLEngine.live_benchmarks_applied,
        "live_survey_benchmarks_sample_size": MLEngine.live_benchmarks_sample_size,
        "live_survey_benchmarks_computed_at": MLEngine.live_benchmarks_computed_at,
        "ollama_reachable": ollama_reachable,
        "ollama_error": ollama_error,
```

```
        "ollama_categorize_model": ollama_client.CATEGORIZE_MODEL,
        "ollama_chat_model": ollama_client.CHAT_MODEL,
    }

# — Phase 2: local-Ollama-backed features —
# Every call here goes to http://localhost:11434 only (app/ml/ollama_client.py)
# — nothing in this block, or anything it calls, is allowed to reach an
# external LLM API. Categorization is open to any authenticated user (it
# doesn't touch anyone's stored data, just a description string handed in
# the request); insights/chat both read the target user's own Firestore
# transactions server-side, so both are gated by _require_self_or_superuser
# to keep that boundary matching what firestore.rules already enforces for
# direct client reads of the same collection.

@app.post("/api/v1/ml/categorize", response_model=CategorizeResponse, tags=["LLM"])
async def categorize_transaction(
    req: CategorizeRequest,
    current_user: dict = Depends(verify_firebase_token)
):
    """
    Free-text transaction description -> best-fit category id + confidence,
    via the local qwen2.5:1.5b model. Staged honesty: a low-confidence or
    unparseable result comes back with category_id=null, needs_review=true
    – never forced into a category the model wasn't actually sure about.
    """
    result = llm_features.categorize_transaction(req.description, req.amount)
    return CategorizeResponse(
        category_id=result["category_id"],
        category_name=result["category_name"],
        confidence=result["confidence"],
        needs_review=result["needs_review"],
    )

@app.post("/api/v1/ml/insights", response_model=InsightResponse, tags=["LLM"])
async def get_spend_insight(
    req: InsightRequest,
    current_user: dict = Depends(verify_firebase_token)
):
    """
    Short, human-readable nudge about the caller's own recent category-level
    spend (e.g. "You've spent 40% more on snacks this week than usual"),
    generated by the local qwen2.5:7b-instruct model. Every number in the
    sentence is computed here in Python from real Firestore data first –
    the model is only ever asked to phrase numbers it was handed, never to
    produce or adjust them (see llm_features.build_insight).
    """
```

```

    _require_self_or_superuser(current_user, req.user_id)
    db = get_firestore_client()
    if db is None:
        raise HTTPException(status_code=503, detail="Firestore unavailable – cannot compute insight"
)
    result = llm_features.build_insight(db, req.user_id)
    return InsightResponse(user_id=req.user_id, insight=result["insight"])

@app.post("/api/v1/ml/chat", response_model=ChatResponse, tags=["LLM"])
async def chat_with_assistant(
    req: ChatRequest,
    current_user: dict = Depends(verify_firebase_token)
):
    """
    Natural-language Q&A about the caller's own spending. Fetches their real
    recent transactions first and hands the model a computed numeric
    summary as context (see llm_features.answer_chat) – the model answers
    from that summary, never from nothing and never from raw unrestricted
    Firestore access.
    """
    _require_self_or_superuser(current_user, req.user_id)
    db = get_firestore_client()
    if db is None:
        raise HTTPException(status_code=503, detail="Firestore unavailable – cannot fetch spending d
ata")
    result = llm_features.answer_chat(db, req.user_id, req.message)
    return ChatResponse(
        user_id=req.user_id,
        answer=result["answer"],
        grounded_on_transaction_count=result["grounded_on_transaction_count"],
    )

# — Step 10: Survey ETL / Stage-1 training stats / superadmin data config —
# All three endpoints below are superadmin-only (require_superuser) – the
# raw stats and the ability to change what feeds the ML heuristic aren't
# public. See backend/app/ml/survey_etl.py for the actual computation; these
# are thin wrappers that make it callable from the admin panel without a
# redeploy, which was the explicit point of this pass.

@app.get("/api/v1/admin/data-config", tags=["Superadmin", "Training"])
async def get_data_config(admin_user: dict = Depends(require_superuser)):
    """Current admin/dataConfig (income band cutoffs, per-category weights,
    outlier handling, confidence threshold) – defaults filled in if the doc
    doesn't exist yet, so the admin panel always has something sane to render."""
    db = get_firestore_client()
    return survey_etl.load_data_config(db)

```

```
@app.put("/api/v1/admin/data-config", tags=["Superadmin", "Training"])
async def update_data_config(
    config: Dict[str, Any] = Body(...),
    admin_user: dict = Depends(require_superuser)
):
    """
    Saves a new admin/dataConfig and immediately recomputes Stage-1 stats
    against it – this IS the "changing these values triggers a recompute,
    not a redeploy" mechanism from the brief. Partial updates are fine
    (missing keys keep their existing/default value); unknown keys are
    dropped rather than silently stored, since a typo'd key would otherwise
    just be ignored forever without anyone noticing.
    """
    db = get_firestore_client()
    if db is None:
        raise HTTPException(status_code=503, detail="Firestore unavailable – cannot save config or r
ecompute")

    merged = survey_etl.load_data_config(db)
    for key in survey_etl.DEFAULT_DATA_CONFIG:
        if key in config:
            merged[key] = config[key]
    db.collection("admin").document("dataConfig").set(merged)

    stats = survey_etl.run_etl(db)
    MLEngine.apply_live_benchmarks(stats)
    return {"config": merged, "recomputedStats": stats}

@app.post("/api/v1/admin/recompute-benchmarks", tags=["Superadmin", "Training"])
async def recompute_benchmarks(admin_user: dict = Depends(require_superuser)):
    """Re-runs the Stage-1 ETL against current admin/dataConfig and whatever
    survey_responses exist right now (e.g. to pick up new submissions without
    also changing config), and re-applies the result to the running MLEngine."""
    db = get_firestore_client()
    if db is None:
        raise HTTPException(status_code=503, detail="Firestore unavailable – cannot recompute")
    stats = survey_etl.run_etl(db)
    MLEngine.apply_live_benchmarks(stats)
    return stats

@app.get("/api/v1/admin/training-insights", tags=["Superadmin", "Training"])
async def get_training_insights(admin_user: dict = Depends(require_superuser)):
    """
    Everything the Training Insights admin screen needs in one call: the
    latest computed Stage-1 stats (per-category-per-income-band median/IQR/
```

```

sample size), a sample-size trend history, and the population-level
dot-graph clustering preview (item 4 in the brief). Computes fresh from
current Firestore data on every call rather than only ever showing a
stale cached snapshot – this endpoint is admin-only and low-traffic by
nature, so the recompute cost isn't a real concern.
"""

db = get_firestore_client()
if db is None:
    raise HTTPException(status_code=503, detail="Firestore unavailable")

stats = survey_etl.run_etl(db)
MLEngine.apply_live_benchmarks(stats)
history = survey_etl.load_benchmark_history(db)
config = stats["configUsed"]
responses = survey_etl.fetch_survey_responses(db)
dot_graph = survey_etl.generate_population_dot_graph(responses, config)

return {
    "stats": stats,
    "history": history,
    "populationDotGraph": dot_graph,
}

# — Step 14: public, unauthenticated survey stats for antaraweb —
# Powers the live figures on the antara.money research-paper site (separate
# `antaraweb` repo – plain HTML/JS, fetches this directly from the browser).
# Deliberately outside the /api/v1/admin/* group above: no require_superuser
# dependency, since this is aggregates-only over an already-anonymous survey
# (see survey_etl.py's module docstring) – nothing here is more sensitive
# than what /api/v1/admin/training-insights already exposes to admins, just
# reshaped into a smaller, public-safe view via build_public_stats_payload.
#
# CORS: no route-level config needed or possible with FastAPI's
# CORSMiddleware – it's applied once, globally, above (line ~56), so every
# route including this one is already covered by ALLOWED_ORIGINS. Verified
# (not assumed) that "https://antara.money" is still in that list from Step
# 9 – it is (see ALLOWED_ORIGINS above) – so no addition was needed here.
@app.get("/api/v1/public/survey-stats", tags=["Public"])
async def get_public_survey_stats():
    """
    Unauthenticated, read-only aggregate view of the Stage-1 survey stats:
    per-category median/Q1/Q3/n/confidence, income-band respondent counts,
    and per-respondent archetype match percentages (no PII – see
    survey_etl.build_public_stats_payload). Reuses Step 10's survey_etl
    computation rather than duplicating it.
    """

```

```
    db = get_firestore_client()
    if db is None:
        raise HTTPException(status_code=503, detail="Firestore unavailable")
    return survey_etl.build_public_stats_payload(db)

if __name__ == "__main__":
    port = int(os.getenv("PORT", 8001))
    uvicorn.run("app.main:app", host="0.0.0.0", port=port, reload=True)
```

End of the first 10 pages.

Pages 11 through 274 of the full 284-page concatenation are omitted from this excerpt per Copyright Office practice for a representative sample.

The last 10 pages of the concatenation follow.

```

        block = {}
        for cat_id in SURVEY_CATEGORY_KEYS:
            stats = _describe(cat_values.get(cat_id, []), config)
            weight = float(weights.get(cat_id, 1.0))
            if stats["insufficientData"]:
                raw_pct = 0.0
            else:
                cat_total = sum(cat_values.get(cat_id, []))
                raw_pct = (cat_total / grand_total) if grand_total > 0 else 0.0
            stats["rawBenchmarkPct"] = round(raw_pct, 4)
            stats["categoryWeight"] = weight
            stats["adjustedBenchmarkPct"] = _blend_benchmark(raw_pct, weight, len(SURVEY_CATEGORY_KEYS))

            stats["confidenceTier"] = (
                "early_estimate" if stats["insufficientData"] else _confidence_tier(stats["sampleSizeUsed"], config)
            )
            block[cat_id] = stats
        return block

    overall = build_category_block(values["__overall__"])
    by_income_band = {
        label: {"respondentCount": band_counts.get(label, 0), "categories": build_category_block(values.get(label, {}))}
        for label in band_labels
        if band_counts.get(label, 0) > 0 # don't report a phantom empty band
    }

    return {
        "computedAt": computed_at,
        "sampleSize": n_total,
        "grandTotalReported": round(grand_total, 2),
        "configUsed": config,
        "overall": overall,
        "byIncomeBand": by_income_band,
        "unmappedCategories": unmapped_categories, # non-empty here is a real signal something needs attention
    }

def save_benchmarks(db, stats: Dict[str, Any]) -> None:
    if db is None:
        return
    db.collection("admin").document("categoryBenchmarks").set(stats)
    # Append a lightweight trend point (computedAt + sampleSize only, not the
    # full stats blob) so the Training Insights view has something real to

```

```
# plot as more survey responses come in, instead of a single static number.
db.collection("admin").document("categoryBenchmarks").collection("history").add(
    {"computedAt": stats["computedAt"], "sampleSize": stats["sampleSize"]}
)

def load_benchmarks(db) -> Optional[Dict[str, Any]]:
    if db is None:
        return None
    try:
        snap = db.collection("admin").document("categoryBenchmarks").get()
        return snap.to_dict() if snap.exists else None
    except Exception as e:
        print(f"[!] Warning: could not load admin/categoryBenchmarks: {e}")
        return None

def load_benchmark_history(db, limit: int = 50) -> List[Dict[str, Any]]:
    if db is None:
        return []
    try:
        docs = (
            db.collection("admin")
            .document("categoryBenchmarks")
            .collection("history")
            .order_by("computedAt")
            .limit(limit)
            .stream()
        )
        return [d.to_dict() for d in docs]
    except Exception as e:
        print(f"[!] Warning: could not load benchmark history: {e}")
        return []

def generate_population_dot_graph(responses: List[Dict[str, Any]], config: Dict[str, Any]) -> Dict[str, Any]:
    """Population-level dot-graph preview (Step 10 item 4) – same visual
    language as the per-user dot graph in engine.py's generate_dot_graph
    (reuses the exact same PEER_ARCHETYPES definitions and node/link shape),
    but every node here is one survey RESPONDENT rather than one category.
    Explicitly a sanity check on the archetype-clustering approach against
    real population data before it's ever trusted on a single live user's
    transaction history – not a trained clustering model. With the current
    tiny sample size this will look sparse; that's honest, not a bug, and
    `sampleSize` is always attached so nothing downstream can present it as
```

```

more than it is.
"""

cat_keys = SURVEY_CATEGORY_KEYS
nodes: List[Dict[str, Any]] = []
links: List[Dict[str, Any]] = []

# Fixed archetype anchor nodes, same 5 as the per-user dot graph, arranged
# in an outer ring so respondent nodes have a stable reference frame.
arc_radius = 260.0
angle_step = (2 * math.pi) / len(PEER_ARCHETYPES)
archetype_positions = {}
for j, arc in enumerate(PEER_ARCHETYPES):
    ax = math.cos(j * angle_step) * arc_radius
    ay = math.sin(j * angle_step) * arc_radius
    archetype_positions[arc["id"]] = (ax, ay)
    nodes.append({
        "id": f"archetype_{arc['id']}",
        "label": arc["name"],
        "type": "archetype_center",
        "size": 22,
        "color": arc["color"],
        "x": round(ax, 2),
        "y": round(ay, 2),
        "metadata": {"description": arc["description"]},
    })

respondent_count = 0
for i, r in enumerate(responses):
    spend = r.get("category_spend") or {}
    vec = [float(spend.get(k, 0) or 0) for k in cat_keys]
    total = sum(vec)
    if total <= 0:
        continue # a response with literally no spend logged has no pattern to place
    respondent_count += 1

    best_arc, best_sim = PEER_ARCHETYPES[0], -1.0
    for arc in PEER_ARCHETYPES:
        arc_vec = [arc["weights"].get(k, 0.01) for k in cat_keys]
        arc_norm, user_norm = np.linalg.norm(arc_vec), np.linalg.norm(vec)
        sim = float(np.dot(vec, arc_vec) / (arc_norm * user_norm)) if (arc_norm > 0 and user_norm > 0) else 0.0
        if sim > best_sim:
            best_sim, best_arc = sim, arc

    ax, ay = archetype_positions[best_arc["id"]]
    # Closer to its matched archetype = stronger match; jittered by

```

```

# respondent index (deterministic, not random) so exact ties don't
# visually stack on the same point.
pull = max(0.15, min(0.85, best_sim))
jitter_angle = (i * 47) % 360 * (math.pi / 180)
jitter_r = 26.0
nx = ax * pull + math.cos(jitter_angle) * jitter_r
ny = ay * pull + math.sin(jitter_angle) * jitter_r

band = classify_income_band((r.get("demographics") or {}).get("pocket_money_range"), config)
node_id = f"respondent_{i}"
nodes.append({
    "id": node_id,
    "label": f"Respondent {i + 1} ({band})",
    "type": "survey_respondent",
    "size": 10 + min(14, math.sqrt(total) / 4),
    "color": best_arc["color"],
    "x": round(nx, 2),
    "y": round(ny, 2),
    "metadata": {
        "totalReported": total,
        "incomeBand": band,
        "bestMatchArchetype": best_arc["name"],
        "similarityPct": round(best_sim * 100, 1),
    },
})
links.append({
    "source": node_id,
    "target": f"archetype_{best_arc['id']}",
    "strength": round(max(0.1, best_sim), 3),
    "distance": round(120 * (1 - pull) + 40, 1),
    "type": "similarity",
})

return {
    "sampleSize": len(responses),
    "respondentsPlotted": respondent_count,
    "nodes": nodes,
    "links": links,
    "note": (
        "Sanity-check preview only – clusters respondents against the same fixed archetype "
        "definitions the per-user dot graph uses, not a trained model. Treat any apparent "
        "clustering as illustrative until sampleSize is much larger."
    ),
}

```

```

def _archetype_slug(archetype_id: str) -> str:
    """archetype_gamer_foodie" -> "gamer-foodie" – the compact id used by
    the public payload and by antaraweb's Fig. 4 anchors, derived from the
    single PEER_ARCHETYPES id rather than maintained as a second list."""
    return archetype_id.removeprefix("archetype_").replace("_", "-")

def build_public_stats_payload(db) -> Dict[str, Any]:
    """Shapes Step 10's Stage-1 stats + population dot-graph into the
    minimal, PII-free aggregate payload served by the public
    GET /api/v1/public/survey-stats endpoint (Step 14 – see backend/app/
    main.py). Reuses compute_stats/generate_population_dot_graph directly;
    this function is a view over their output, not a second pipeline.

    Safe to expose unauthenticated: survey_responses carries no name/email/
    uid (confirmed in this module's docstring above), and every field below
    is either a category aggregate (median/q1/q3/n/confidence) or a single
    per-respondent match percentage against a fixed archetype – never a raw
    response, a respondent index, or anything else identifying.
    """

    config = load_data_config(db)
    responses = fetch_survey_responses(db)
    stats = compute_stats(responses, config)
    dot_graph = generate_population_dot_graph(responses, config)

    categories = {
        cat_id: {
            "median": None if c["insufficientData"] else c["median"],
            "q1": None if c["insufficientData"] else c["q1"],
            "q3": None if c["insufficientData"] else c["q3"],
            "n": c["sampleSizeUsed"],
            "outliersRemoved": c.get("outliersRemoved", 0),
            "confidenceTier": c["confidenceTier"],
        }
        for cat_id, c in stats["overall"].items()
    }

    # byIncomeBand keys are the configured band labels ("Low"/"Mid"/"High"
    # by default, or "Unknown") – lowercased for the public payload; a band
    # with zero respondents is already absent from byIncomeBand (see
    # compute_stats), so nothing here fabricates a phantom "unknown": 0.
    income_bands = {
        label.lower(): block["respondentCount"]
        for label, block in stats["byIncomeBand"].items()
    }

```

```
archetype_clusters: Dict[str, Any] = {
    _archetype_slug(arc["id"]): {"count": 0, "respondents": []}
    for arc in PEER_ARCHETYPES
}
id_by_name = {arc["name"]: arc["id"] for arc in PEER_ARCHETYPES}
for node in dot_graph["nodes"]:
    if node["type"] != "survey_respondent":
        continue
    meta = node["metadata"]
    slug = _archetype_slug(id_by_name[meta["bestMatchArchetype"]])
    archetype_clusters[slug]["respondents"].append(
        {"matchPct": round(meta["similarityPct"] / 100, 4)}
    )
for block in archetype_clusters.values():
    block["count"] = len(block["respondents"])

return {
    "sampleSize": stats["sampleSize"],
    "lastUpdated": stats["computedAt"],
    "categories": categories,
    "incomeBands": income_bands,
    "archetypeClusters": archetype_clusters,
}

def run_etl(db) -> Dict[str, Any]:
    """End-to-end: load config, fetch responses, compute, persist. Called
    both at backend startup and on-demand from the superadmin recompute
    endpoint – the whole point (per the brief) is that changing dataConfig
    triggers this, not a redeploy."""
    config = load_data_config(db)
    responses = fetch_survey_responses(db)
    stats = compute_stats(responses, config)
    save_benchmarks(db, stats)
    return stats
```

```
from typing import List, Optional, Dict, Any
from pydantic import BaseModel, Field
from datetime import datetime

class TransactionItem(BaseModel):
    id: Optional[str] = None
    amount: float = Field(..., gt=0, description="Amount in INR (₹)")
    category: str = Field(..., description="Category ID (e.g. food-delivery, gaming, subscriptions)")
)
    subcategory: Optional[str] = None
    note: Optional[str] = None
    timestamp: datetime = Field(default_factory=datetime.utcnow)
    source: Optional[str] = "manual"

class SpendPredictRequest(BaseModel):
    user_id: str
    transactions: List[TransactionItem]
    monthly_budget: Optional[float] = 5000.0
    period_days: Optional[int] = 30

class CategoryForecast(BaseModel):
    category_id: str
    category_name: str
    predicted_spend: float
    confidence: float
    historical_spend: float
    trend_pct: float
    risk_level: str # 'low', 'medium', 'high'
    is_heuristic: bool = True

class SpendPredictResponse(BaseModel):
    user_id: str
    predicted_total_spend: float
    current_burn_rate_daily: float
    predicted_burn_rate_daily: float
    projected_days_until_budget_exhaustion: Optional[float]
    top_risk_categories: List[str]
    category_breakdown: List[CategoryForecast]
    smart_insights: List[str]
    is_cold_start: bool
    model_mode: str # 'HEURISTIC_COLD_START' | 'TRAINED_EMBEDDING_V1'
    data_days_logged: int
    data_points_count: int
    confidence_score: float
    last_retrained_at: Optional[datetime] = None
    generated_at: datetime = Field(default_factory=datetime.utcnow)
```

```
class GraphNode(BaseModel):
    id: str
    label: str
    type: str # 'user', 'category', 'peer_cluster', 'transaction_spike'
    size: float
    color: str
    category_id: Optional[str] = None
    amount: Optional[float] = 0.0
    x: Optional[float] = None
    y: Optional[float] = None
    metadata: Dict[str, Any] = Field(default_factory=dict)

class GraphLink(BaseModel):
    source: str
    target: str
    strength: float
    distance: float
    type: str = "gravity" # 'gravity', 'similarity', 'co_occurrence'

class DotGraphResponse(BaseModel):
    user_id: str
    archetype: str
    archetype_description: str
    is_cold_start: bool
    model_mode: str
    data_days_logged: int
    confidence_score: float
    embedding: List[float]
    nodes: List[GraphNode]
    links: List[GraphLink]
    peer_archetypes: List[Dict[str, Any]]
    last_retrained_at: Optional[datetime] = None
    generated_at: datetime = Field(default_factory=datetime.utcnow)

class LearningCurvePoint(BaseModel):
    date: str # ISO calendar date – one of the user's own real logged days
    confidence: float
    model_mode: str # 'HEURISTIC_COLD_START' | 'TRAINED_EMBEDDING_V1'
    is_cold_start: bool
    tx_count: int
    active_days: int

class LearningCurveResponse(BaseModel):
    user_id: str
    points: List[LearningCurvePoint]
```

```
generated_at: datetime = Field(default_factory=datetime.utcnow)

# — "Instances" — user-pinned budget allocation with ML-suggested remainder —

class AllocateBudgetRequest(BaseModel):
    user_id: str
    transactions: List[TransactionItem]
    monthly_budget: float = Field(..., gt=0)
    pinned: Dict[str, float] = Field(default_factory=dict, description="category_id -> user-pinned a  
mount in INR")

class CategoryAllocation(BaseModel):
    category_id: str
    category_name: str
    is_pinned: bool
    amount: float
    is_early_estimate: bool

class AllocateBudgetResponse(BaseModel):
    user_id: str
    allocations: List[CategoryAllocation]
    pinned_total: float
    remaining_after_pinned: float
    over_allocated: bool
    generated_at: datetime = Field(default_factory=datetime.utcnow)

class HealthResponse(BaseModel):
    status: str
    service: str
    port: int
    version: str
    timestamp: datetime = Field(default_factory=datetime.utcnow)

# — Phase 2: local-Ollama-backed features (see app/ml/ollama_client.py /  
# app/ml/llm_features.py) —

class CategorizeRequest(BaseModel):
    description: str = Field(..., min_length=1, max_length=300)
    amount: Optional[float] = None

class CategorizeResponse(BaseModel):
    category_id: Optional[str]
    category_name: Optional[str]
    confidence: float
    needs_review: bool
```

```
class InsightRequest(BaseModel):
    user_id: str

class InsightResponse(BaseModel):
    user_id: str
    insight: Optional[str]
    generated_at: datetime = Field(default_factory=datetime.utcnow)

class ChatRequest(BaseModel):
    user_id: str
    message: str = Field(..., min_length=1, max_length=500)

class ChatResponse(BaseModel):
    user_id: str
    answer: str
    grounded_on_transaction_count: int
    generated_at: datetime = Field(default_factory=datetime.utcnow)
```